

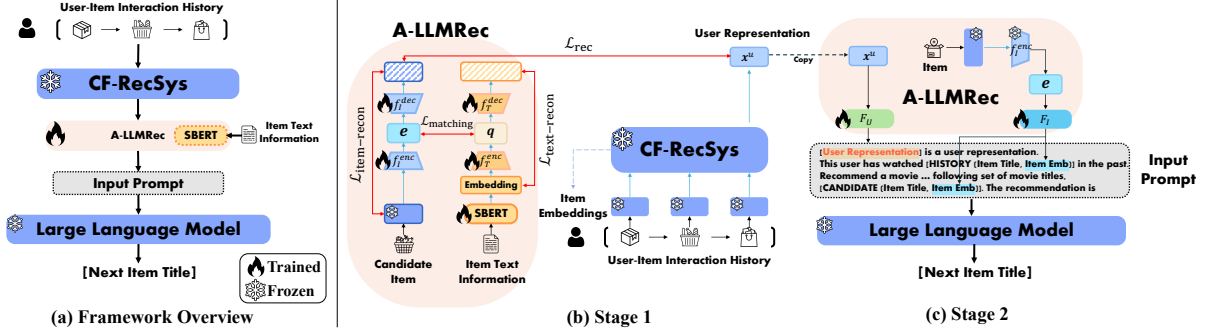


Figure 2: (a) is the overview of A-LLMRec. (b) and (c) are the detailed architecture of Stage 1 and Stage 2, respectively.

readily applied to non-sequential recommendation tasks by simply replacing the backbone CF-RecSys, e.g., from SASRec [20] (sequential) to NCF [15] (non-sequential), which will be demonstrated in the experiments (Section 5.4.3).

4 PROPOSED METHOD: A-LLMREC

In this section, we propose A-LLMRec, a novel LLM-based recommender framework that aligns a frozen pre-trained collaborative filtering recommender (CF-RecSys) with a frozen LLM aiming to enhance the recommendation performance not only in the cold scenario but also in the warm scenario. To bridge the modality gap, A-LLMRec aligns collaborative knowledge of the CF-RecSys with the token space of the LLM. Our approach involves two pre-training stages: (1) Aligning collaborative and textual knowledge with a frozen CF-RecSys (Section 4.1), and (2) Recommendation stage with a frozen LLM (Section 4.2) in which the joint collaborative and textual knowledge is projected onto the LLM.

4.1 Alignment between Collaborative and Textual Knowledge (Stage-1)

In this section, we introduce how to align the item embeddings from a frozen CF-RecSys with their associated text information to capture both collaborative and textual knowledge. We employ a pre-trained Sentence-BERT (SBERT) [35] model, which is fine-tuned during training, to extract text embeddings from textual information associated with items³. Then, we introduce two encoders, i.e., item encoder f_I^{enc} and text encoder f_T^{enc} , each containing a 1-layer Multi-Layer Perceptron (MLP), to align the item embeddings from a frozen CF-RecSys with the text embeddings from SBERT. Given an item i , the item encoder $f_I^{enc} : \mathbb{R}^d \rightarrow \mathbb{R}^{d'}$ encodes an item embedding $E_i \in \mathbb{R}^d$ into a latent item embedding $e_i \in \mathbb{R}^{d'}$, i.e., $e_i = f_I^{enc}(E_i)$, while the text encoder $f_T^{enc} : \mathbb{R}^{768} \rightarrow \mathbb{R}^{d'}$ encodes a text embedding $Q_i \in \mathbb{R}^{768}$ from SBERT, whose output dimension size is 768, into a latent text embedding $q_i \in \mathbb{R}^{d'}$, i.e., $q_i = f_T^{enc}(Q_i)$. Then, we perform latent space matching between item embeddings and text embeddings as follows:

$$\begin{aligned} \mathcal{L}_{matching} &= \mathbb{E}_{S^u \in S} \left[\mathbb{E}_{i \in S^u} [MSE(e_i, q_i)] \right] \\ &= \mathbb{E}_{S^u \in S} \left[\mathbb{E}_{i \in S^u} [MSE(f_I^{enc}(E_i), f_T^{enc}(Q_i))] \right] \end{aligned} \quad (2)$$

³Although using a larger language model, such as OPT [52] and LLaMA [42], would further enhance the quality of the text embeddings, we adopt SBERT for efficiency.

where $Q_i = SBERT(\text{"Title : } t^i, \text{Description : } d^i\text{"})$ denotes the encoded representation of item text (i.e., item title and description) by SBERT, and MSE is the mean squared error loss. That is, we match the item embeddings from a frozen CF-RecSys and the text embeddings from SBERT in the latent space of the encoders, so as to align the semantics of items and their associated texts for later use in the LLM.

4.1.1 Avoiding Over-smoothed Representation. On the other hand, simply optimizing the latent space matching loss defined in Equation 2 would result in over-smoothed representations, i.e., the encoders would be trained to produce similar outputs (i.e., $e_i \approx q_i$) to minimize $\mathcal{L}_{matching}$. In an extreme case, the output of the encoders would be collapsed to a trivial representation by assigning their weights to all zeros. Hence, to prevent this issue and preserve the original information of the item and its associated text embedding, we add a decoder to each of the encoders and introduce reconstruction losses as follows:

$$\mathcal{L}_{item-recon} = \mathbb{E}_{S^u \in S} \left[\mathbb{E}_{i \in S^u} [MSE(E_i, f_I^{dec}(f_I^{enc}(E_i)))] \right] \quad (3)$$

$$\mathcal{L}_{text-recon} = \mathbb{E}_{S^u \in S} \left[\mathbb{E}_{i \in S^u} [MSE(Q_i, f_T^{dec}(f_T^{enc}(Q_i)))] \right] \quad (4)$$

where f_I^{dec} and f_T^{dec} are the decoders added to the encoders f_I^{enc} and f_T^{enc} , respectively. In Section 5.3.1, we empirically demonstrate the benefit of introducing the reconstruction losses.

4.1.2 Recommendation Loss. Besides aligning the collaborative knowledge from the user-item interactions with the textual knowledge from the associated text information, we introduce a recommendation loss to explicitly incorporate the collaborative knowledge, while informing the model about the recommendation task. Specifically, the recommendation loss is defined as follows [20]:

$$\begin{aligned} \mathcal{L}_{rec} &= - \sum_{S^u \in S} \left[\log(\sigma(s(x_{|S^u|-1}^u, f_I^{dec}(f_I^{enc}(E_{i_{|S^u|}^u)))) \right. \\ &\quad \left. + \log(1 - \sigma(s(x_{|S^u|-1}^u, f_I^{dec}(f_I^{enc}(E_{i_{|S^u|}^{u,-}})))) \right] \end{aligned} \quad (5)$$

where $x_{|S^u|-1}^u = CF-RecSys(S_{1:|S^u|-1}^u) \in \mathbb{R}^d$ is the user representation extracted from the collaborative filtering recommender system, i.e., CF-RecSys, obtained after the user u has interacted with the last item in the sequence $S_{1:|S^u|-1}^u$, and $E_{i_{|S^u|}^u} \in \mathbb{R}^d$ is the embedding of a positive item of $i_{|S^u|}^u$, i.e., $i_{|S^u|}^u$, and $s(a, b)$ is a dot product between a and b .

4.1.3 Final Loss of Stage-1. Finally, the final objective of Stage-1, i.e., $\mathcal{L}_{\text{stage-1}}$, is the sum of the matching loss defined in Equation 2, reconstruction losses defined in Equation 3 and 4, and recommendation loss in Equation 5:

$$\mathcal{L}_{\text{stage-1}} = \mathcal{L}_{\text{matching}} + \alpha \mathcal{L}_{\text{item-recon}} + \beta \mathcal{L}_{\text{text-recon}} + \mathcal{L}_{\text{rec}} \quad (6)$$

where α and β are the coefficients that control the importance of each term. Note that for efficiency in training, we only considered the last item in S^u for each user u to minimize $\mathcal{L}_{\text{stage-1}}$. However, considering all items in the sequence further enhances the recommendation performance, which will be shown in Section 5.4.2.

4.1.4 Joint Collaborative-Text Embedding. Having trained the autoencoder based on Equation 6, we consider $\mathbf{e}_i = f_I^{\text{enc}}(\mathbf{E}_i)$ as the joint collaborative-text embedding (shortly joint embedding) of item i , which will be passed to the LLM as input. The joint embedding introduces the collaborative and textual knowledge to LLMs, which will be described in Section 4.2.

It is important to note that when encountering new items that have not been seen during the training of the collaborative filtering recommender, we can instead rely on the text encoder f_T^{enc} to extract the joint collaborative-text embedding, i.e., $\mathbf{q}_i = f_T^{\text{enc}}(\mathbf{Q}_i)$. Since the two encoders f_I^{enc} and f_T^{enc} are jointly trained to match their latent spaces, we expect the joint embedding $\mathbf{q}_i$ to not only capture the textual knowledge but also to implicitly capture the collaborative knowledge. In summary, we use $\mathbf{e}_i = f_I^{\text{enc}}(\mathbf{E}_i)$ as the joint collaborative-text embedding by default, but we use $\mathbf{q}_i = f_T^{\text{enc}}(\mathbf{Q}_i)$ when item i lacks interactions, i.e., cold item, few-shot, and cross-domain scenarios, which will be demonstrated in the experiments in Section 5.2.2, Section 5.2.4, and Section 5.2.5, respectively.

4.2 Alignment between Joint Collaborative-Text Embedding and LLM (Stage-2)

Recall that in Stage-1 we obtained the joint collaborative-text embeddings by aligning the collaborative knowledge with item textual information. Our goal in Stage-2 is to align these joint embeddings with the token space of the LLM (Section 4.2.1), and design a prompt that allows the LLM to solve the recommendation task by leveraging the learned collaborative knowledge (Section 4.2.2). Figure 2 shows the overall architecture of Stage-2. Note that the component trained in Stage-1, which is also utilized in Stage-2, i.e., f_I^{enc} , is frozen in Stage-2.

4.2.1 Projecting collaborative knowledge onto the token space of LLM. We first project the user representations $\mathbf{x}^u \in \mathbb{R}^d$ and the joint collaborative-text embeddings $\mathbf{e}_i \in \mathbb{R}^{d'}$ obtained from Stage-1 onto the token space of LLM, i.e., $\mathbb{R}^{d^{\text{token}}}$. By doing so, we allow the LLM to take them as inputs. More precisely, we introduce two 2-layer MLPs, i.e., $F_U : \mathbb{R}^d \rightarrow \mathbb{R}^{d^{\text{token}}}$ and $F_I : \mathbb{R}^{d'} \rightarrow \mathbb{R}^{d^{\text{token}}}$, to project the user representations and the joint collaborative-text embeddings to the token space of LLM, respectively, as follows:

$$\mathbf{O}_u = F_U(\mathbf{x}^u), \mathbf{O}_i = F_I(\mathbf{e}_i) \quad (7)$$

where $\mathbf{O}_u \in \mathbb{R}^{d^{\text{token}}}$ and $\mathbf{O}_i \in \mathbb{R}^{d^{\text{token}}}$ are the projected embeddings of the representation of user u and the joint collaborative-text embedding of item i , and they can now be used as inputs to LLM prompts, which allow the LLM to perform recommendation without any fine-tuning.

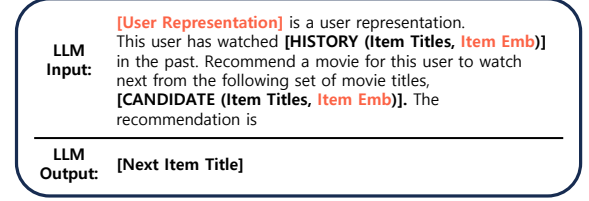


Figure 3: An example prompt of A-LLMRec designed for the Amazon Movies dataset. For other datasets, we keep the same format but adjust the verbs and nouns to fit the context (e.g., ‘watched’ → ‘bought’, ‘movie’ → ‘item’).

4.2.2 Prompt Design for Integrating Collaborative Knowledge. Prompt engineering helps in understanding the capabilities and limitations of LLMs, enabling them to perform complex tasks such as question answering and arithmetic reasoning [4, 46]. Recent studies on LLM-based recommender systems have shown that carefully crafted prompts enhance the performance of LLMs [2, 16, 37]. However, as existing LLM-based recommender systems focus on cold scenarios with few user-item interactions, their prompts mainly consider ways to incorporate modality information (e.g., item description text), while overlooking the collaborative knowledge. To this end, we introduce a novel approach to prompt design for LLM-based recommender system, which combines collaborative knowledge with recommendation instructions (See Figure 3). This is done by directly incorporating user representations $\mathbf{O}_u$ and joint collaborative-text embeddings $\mathbf{O}_i$ into the textual prompts in the token embedding space. In other words, as $\mathbf{O}_u$ and $\mathbf{O}_i$ have been projected into the LLM token space, they can be considered as ordinary tokens used by the LLM and readily incorporated within a prompt. To facilitate the understanding of the LLM regarding the given user, which is crucial for personalized recommendation, we place the projected user representation $\mathbf{O}_u$ at the beginning of the prompt to provide the LLM with the information about users, which is analogous to soft prompts [26]. Moreover, we add the projected joint embedding of an item $\mathbf{O}_i$ next to its title. This structured prompt then serves as an input to the LLM, with the expected output being recommendations tailored to the user. The learning objective of Stage-2 is given as follows:

$$\max_{\theta} \sum_{S^u \in S} \sum_{k=1}^{|y^u|} \log(P_{\theta, \Theta}(y_k^u | p^u, y_{<k}^u)) \quad (8)$$

where θ denotes the learnable parameters of F_U and F_I , Θ is the frozen parameters of LLM, p^u and y^u are the input prompt and the next item title of user u , respectively. y_k^u is the k -th token of y^u and $y_{<k}^u$ represents the tokens before y_k^u . Note that we only use the last item of each user sequence to train Equation 8 for efficiency.

5 EXPERIMENTS

5.1 Experimental Setup

Datasets. For comprehensive evaluations, we used four datasets from Amazon datasets [13, 32], i.e., Movies and TV, Video Games, Beauty, and Toys, which consist of comprehensive textual information including "title" and "description." Note that we deliberately selected datasets with varying statistics in terms of number of users

Table 1: Overall model performance (Hit@1) over various datasets. The best performance is denoted in bold.

	Collaborative filtering				Modality-aware			LLM-based			
	NCF	NextItNet	GRU4Rec	SASRec	MoRec	CTRL	RECFORMER	LLM-Only	TALLRec	MLP-LLM	A-LLMRec
Movies and TV	0.4273	0.5855	0.5215	0.6154	0.4130	0.3467	0.4865	0.0121	0.2345	0.5838	0.6237
Video Games	0.3159	0.4305	0.4026	0.5402	0.4894	0.2354	0.4925	0.0168	0.4403	0.4788	0.5282
Beauty	0.2957	0.4231	0.4131	0.5298	0.4997	0.3963	0.4878	0.0120	0.5542	0.5548	0.5809
Toys	0.1849	0.1415	0.1673	0.2359	0.1728	0.1344	0.2871	0.0141	0.0710	0.3225	0.3336

Table 2: Statistics of the dataset after preprocessing. Avg. Len denotes the average sequence length of users.

Datasets	#Users	#Items	#Interactions.	Avg. Len
Movies and TV	297,498	59,944	3,409,147	11.46
Video Games	64,073	33,614	598,509	8.88
Beauty	9,930	6,141	63,953	6.44
Toys	30,831	61,081	282,213	9.15

and items to conduct an extensive analysis of the models. The statistics for each dataset after preprocessing are presented in Table 2 and we describe details regarding data preprocessing as follows:

- **Movies and TV** To evaluate the models on a large scale, we select about 300K users and 60K items. Following existing studies [20, 51], we removed users and items with fewer than 5 interactions.
- **Video Games** To evaluate the models on moderate-scale data, which is smaller than the Movies and TV dataset, we select about 64K users and 33K items, removing users and items with fewer than 5 interactions, as in the Movies and TV dataset.
- **Beauty** To compose a small and cold dataset, we select about 9K users and 6K items, removing users and items with fewer than 4 interactions. To retain some information from user-item feedback, we categorized user ratings by treating items above 3 as positive and all others including non-interacted items as negative.
- **Toys** For the evaluation of the models where the number of items is larger than number of users, unlike other datasets, we select about 3K users and 6K items, with the number of items being twice as large as the number of users, and remove users and items with fewer than 4 interactions. Similar to the Beauty dataset, to preserve some information from user-item feedback, we categorize positive and negative items with the criterion of rating 3.

Baselines. We compare A-LLMRec with the following baselines that can be categorized into three types: collaborative filtering recommender systems (NCF [15], NextItNet [50], GRU4Rec [17] and SASRec [20]), modality-aware recommender systems (MoRec [51], CTRL [25], and RECFORMER [24]), and LLM-based recommender systems (LLM-Only, TALLRec [2] and MLP-LLM). For more detail regarding the baselines, please refer to Appendix A

Evaluation Setting. We divide user sequences into training, validation, and test sets. For each user sequence, the most recently interacted item, denoted as $i_{|S^u|}^u$, is used as the test set, while the second most recent user interaction item, $i_{|S^u|-1}^u$, is used as the

Table 3: Hyperparameter specifications of A-LLMRec

	Learning rate stage 1	Learning rate stage 2	embedding dim (CF-RecSys) d	embedding dim ($f_{f_i}^{enc}, f_{f_i}^{enc}$) d'	alpha	beta
Movies and TV	0.0001	0.0001	50	128	0.5	0.5
Video Games	0.0001	0.0001	50	128	0.5	0.5
Beauty	0.0001	0.0001	50	128	0.5	0.2
Toys	0.0001	0.0001	50	128	0.5	0.2

validation set. The remaining sequence of items is used as the training set. To evaluate the performance of sequential recommendation models, we add 19 randomly selected non-interacted items to the test set, so that the test set of each user contains 1 positive item and 19 negative items. For quantitative comparison, we employ a widely used metric, Hit Ratio at 1 (Hit@1) for all experiments.

Implementation Details. Although A-LLMRec is model-agnostic, in this work, we adopt OPT-6.7B [52] as the backbone LLM and SASRec [20] as the pre-trained CF-RecSys. For fair comparisons, we also used OPT-6.7B as the backbone LLM for other LLM-based models (i.e., LLM-Only, TALLRec [2] and MLP-LLM). Moreover, we use SASRec as the CF-RecSys in other modality-aware models (i.e., MoRec [51] and CTRL [25]), and fix the dimension of item and model embeddings to 50 for all the methods and datasets. For RECFORMER [24], we follow the paper and employ Longformer [3] as the backbone network. We set the batch size to 128 for all collaborative filtering-based and modality-aware models. Moreover, the batch size is set to 32 for Stage-1 of A-LLMRec, and 4 for MLP-LLM, TALLRec, and Stage-2 of A-LLMRec. We trained Stage-1 of A-LLMRec for 10 epochs, and Stage-2 of A-LLMRec for 5 epochs, and TALLRec is trained for a maximum of 5 epochs. We use the Adam optimizer to train the models in all datasets. For hyperparameters, we tune the model in certain ranges as follows: learning rate η_1, η_2 in $\{0.01, 0.001, 0.0005, 0.0001\}$ for the training stage each, coefficient α, β in $\{0.1, 0.2, 0.5, 0.75, 1.0\}$ for each, we report the best-performing hyper-parameters for each dataset in Table 3. We use four NVIDIA GeForce A6000 48GB for the Movies and TV dataset to train LLM-based models, and one NVIDIA GeForce A6000 48GB for other datasets including LLM-based and other models.

5.2 Performance Comparison

For comprehensive evaluations of A-LLMRec, we perform evaluations under various scenarios, i.e., general scenario (Sec. 5.2.1), cold/warm item scenario (Sec. 5.2.2), cold user scenario (Sec. 5.2.3), few-shot training scenario (Sec. 5.2.4), cross-domain scenario (Sec. 5.2.5).

5.2.1 Overall Performance. The results of the recommendation task on four datasets are given in Table 1. We have the following observations: 1) A-LLMRec outperforms other LLM-based recommender systems that do not consider the collaborative knowledge